

Beyond Scheduling: Architectural Scaling of LLM Inference

Feb. 24th, 2026

Shivam, Dimash, Aman, Leonard



LLM Inference is a Systems Problem

Model size ↑

Context windows ↑

User concurrency ↑

Serving efficiency ↓

Why LLM Serving Is Hard ?

- Context windows → Exploding
- Models → Massive
- Requests → Heterogeneous
- GPUs → Expensive

GPT Gemini
Claude LLaMA
 DeepSeek

Common Issues addressed in both papers

Resource Mismatch

Core issue: Workload variance breaks fixed parallelism.

KV Cache Growth

Core issue: Memory footprint evolves dynamically per request

Phase Interference

Core issue: Mixing Prefill and Decoding on the same GPU creates bottlenecks

Why are GPUs Idle ?

Static Parallelism Assumptions

Most systems assume:

- Fixed tensor parallel degree
- Fixed pipeline stages
- Fixed instance allocation

But:

- Prefill phase $\neq$ Decode phase
- Short decode $\neq$ Long decode
- Resource needs change over time

Core issue: Parallelism is decided once, but workload evolves.

Fundamental Orthogonal Problems

Axis	Problem	What Changes	Bottleneck
Time Variance	Prefill vs Decode Imbalance	Request phase & length	Underutilization
Spatial Distribution	Parallelism & Communication Inefficiency	GPU / Expert placement	Communication

LoongServe solves across **Time axis**

MegaScale-Infer optimizes along **Space axis**



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

LoongServe

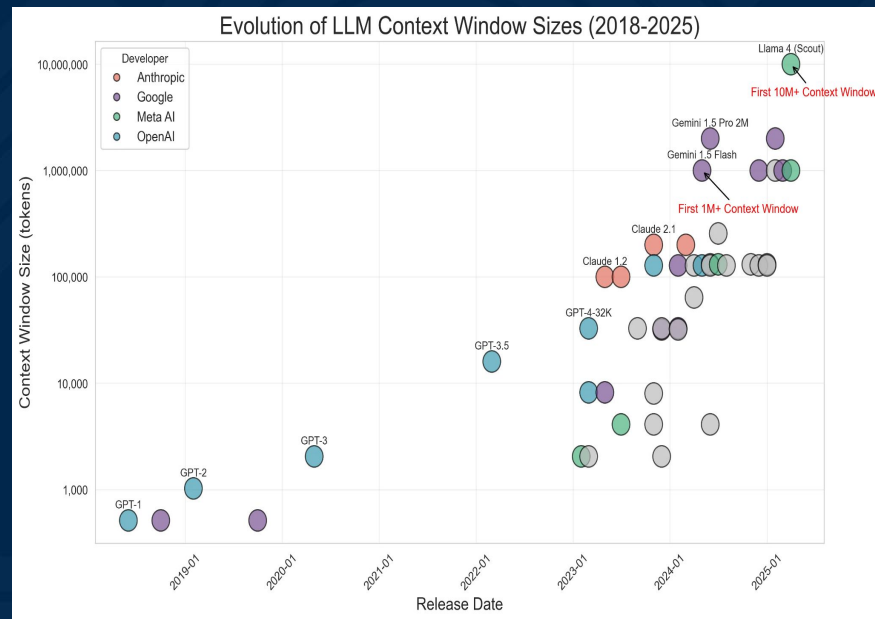
Efficiently Serving Long Context LLMs with
Elastic Sequence Parallelism

The Context Window Revolution & Resource Crisis

The Trend: LLM context windows are expanding to **1M+ tokens** (e.g., Claude-3, Gemini-1.5, LWM).

The Problem: Unprecedented resource demands:

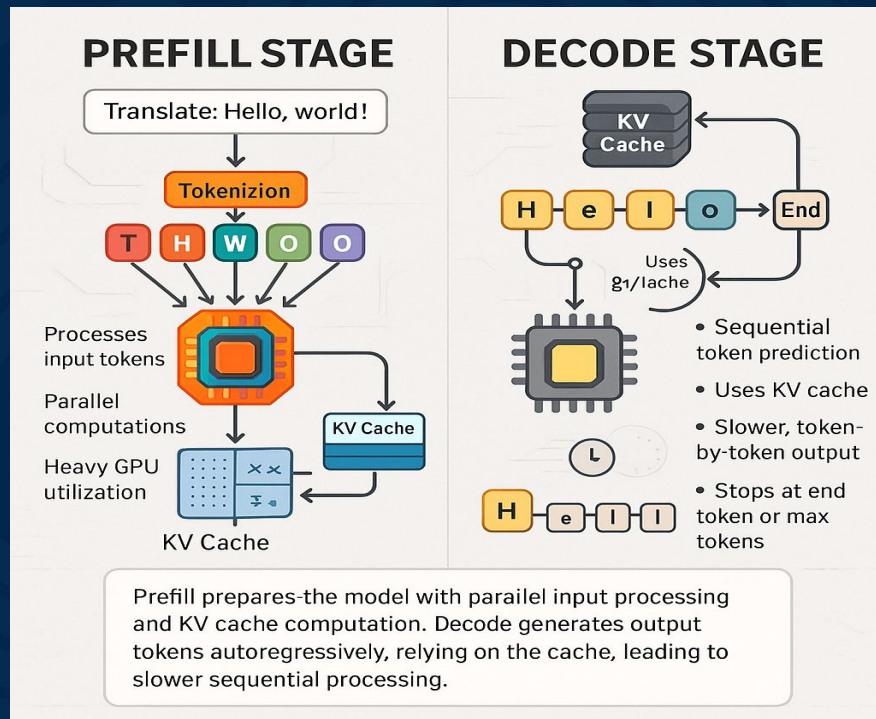
- **Memory:** A 1M context requires **~488GB** for Key-Value (KV) cache alone, far exceeding current single-GPU capacities.
- **Compute:** Attention complexity is **quadratic** to sequence length, making long sequences extremely intensive.



The Process and Variance of LLM Inference

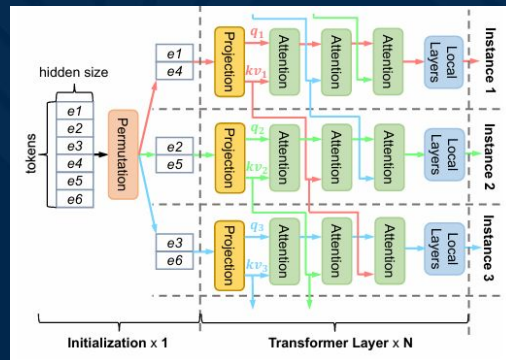
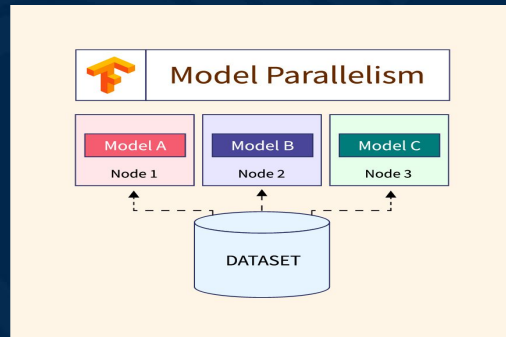
Phase Variance: Inference consists of two distinct phases:

- **Prefill:** Processes all input tokens in one iteration to build the **KV Cache**. Highly compute-intensive.
- **Decoding:** Generates tokens one-by-one, computing KV caches only for new tokens. Primarily memory-bound.



Types of Parallelism

Parallelism Type	Definition	Key Limitation in Serving
Model (Tensor) Parallelism (TP)	Partitions model parameters (weights) across multiple GPUs.	Static Configuration: The Degree of Parallelism (DoP) is fixed at launch; changing it requires a minutes-long system restart.
Sequence Parallelism (SP)	Partitions input sequences across GPUs; instances communicate during attention layers.	Limited Scope: Traditionally only supports the compute-intensive prefill phase with a fixed DoP.

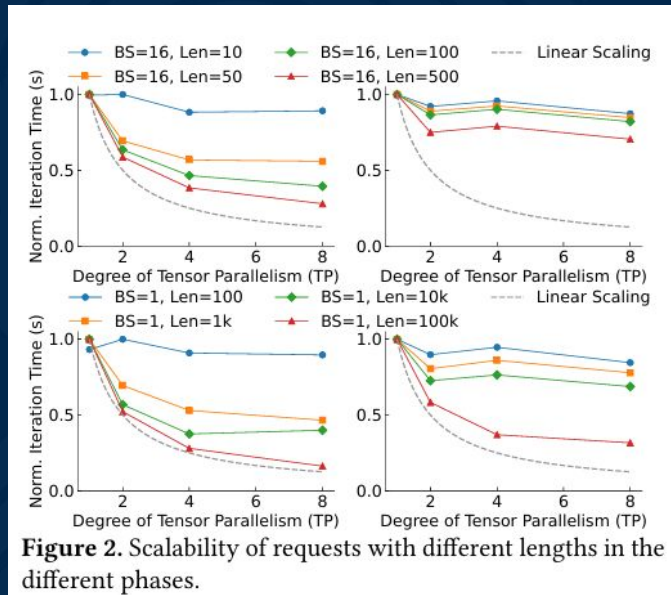


Limitations of Existing Serving Systems

1. Static Parallelism and Resource Mismatch

- **The Constraint:** Traditional systems use **Static Model Parallelism** (e.g., Tensor Parallelism) fixed at startup.
- **The Inefficiency:** One static setting must handle both 1K and 1M token requests.
 - High Degree of Parallelism (DoP) helps long requests but introduces massive communication overhead for short ones.

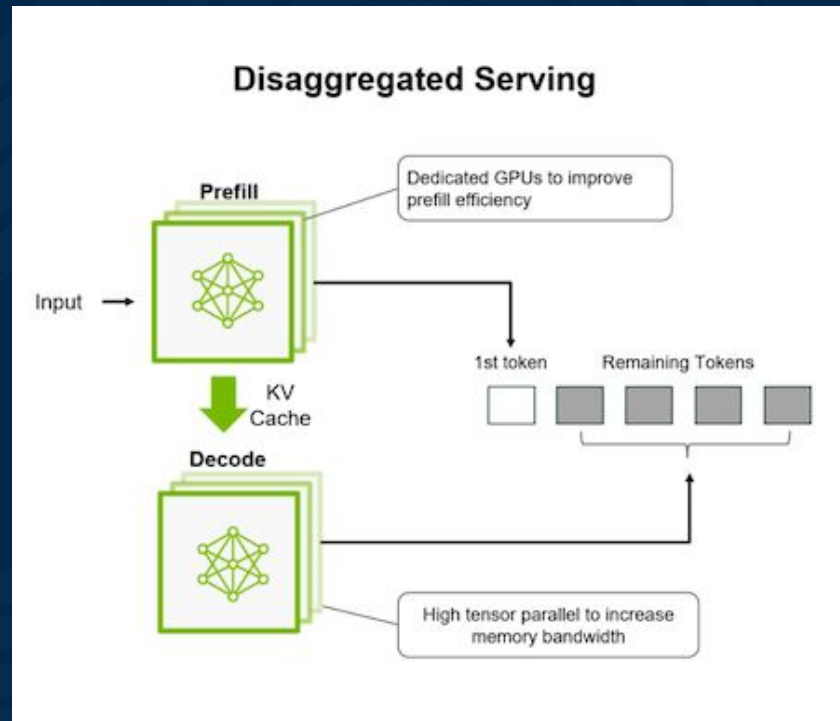
The Wall: Changing the DoP requires a system restart, leading to downtime and lost throughput.



Limitations of Existing Serving Systems

2. The Communication Overhead of Disaggregation

- **The Strategy:** Split GPUs into "Prefill Nodes" and "Decoding Nodes."
- **The Bottleneck: Reactive Migration.**
- **The Cost:** Moving a 488GB KV cache across the network takes seconds—often longer than the inference time itself.
- **Result:** The system spends more time moving data than actually computing answers.



Limitations of Existing Serving Systems

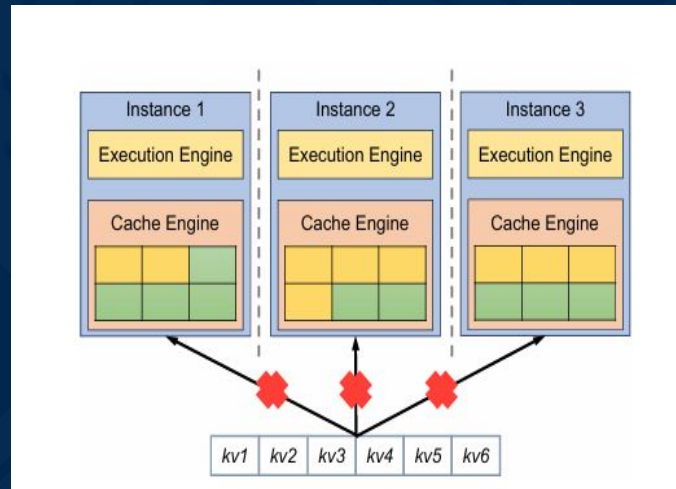
3. GPU Memory Fragmentation

The Issue: Locality Constraint.

The Impact:

- A request's KV cache is traditionally locked to a specific GPU or fixed GPU group.
- If Group A is full and Group B has space, Group B cannot "help" Group A.

Result: The system rejects new requests even though there is free memory across the cluster.



Proposed Sol. : Elastic Sequence Parallelism

Definition: Adjusts the Degree of Parallelism (DoP) at the iteration level for each individual request.

Mechanism: Partitions the input sequence across a variable number of GPUs using Sequence Parallelism (SP) principles.

Dynamic Adaptation:

- **Prefill:** Scales **UP** (High DoP) to utilize massive GPU compute and minimize Time-to-First-Token (TTFT).
- **Decoding:** Scales **DOWN** (Low DoP) to reduce inter-GPU communication overhead and maximize total system throughput.



Potential Drawbacks of ESP

The Elasticity Overhead:

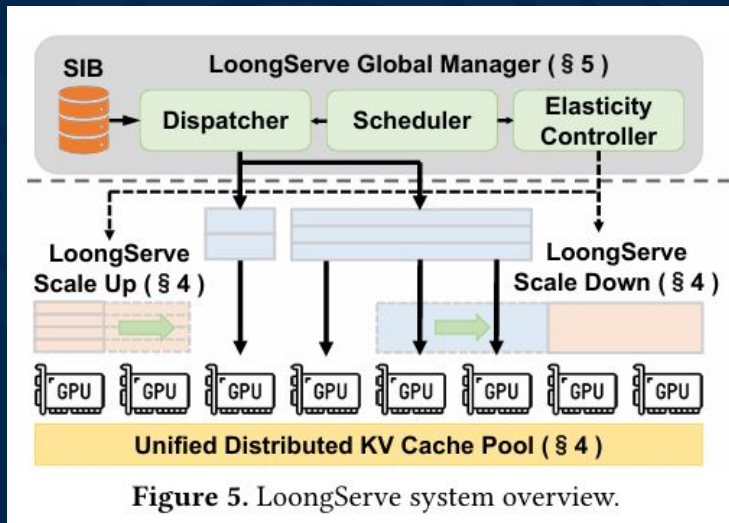
- **The "Migration Tax":** Changing parallelism normally triggers massive data transfers (KV cache migration).
- **The Constraint:** Moving 1M tokens across the network can take seconds, negating the speed gains of scaling.

The Scheduling Complexity:

- **The Multi-Dimensional Decision Space:** The scheduler must simultaneously decide:
 1. Which requests to batch.
 2. How many GPUs to assign (DoP).
 3. Where to place them in the cluster.
- **The Time Wall:** Decisions must be made in milliseconds to match the speed of LLM iterations.

LoongServe Architecture Overview

- **Global Manager:** The centralized controller that receives requests, monitors the status of all GPU resources, and executes the scheduling strategy.
- **Elastic Instances:** Dynamic groups of GPUs that execute the model layers.
- **Unified Distributed KV Cache Pool:** A cluster-wide memory abstraction where KV caches are stored and managed across all available GPU nodes.



Elastic Instances

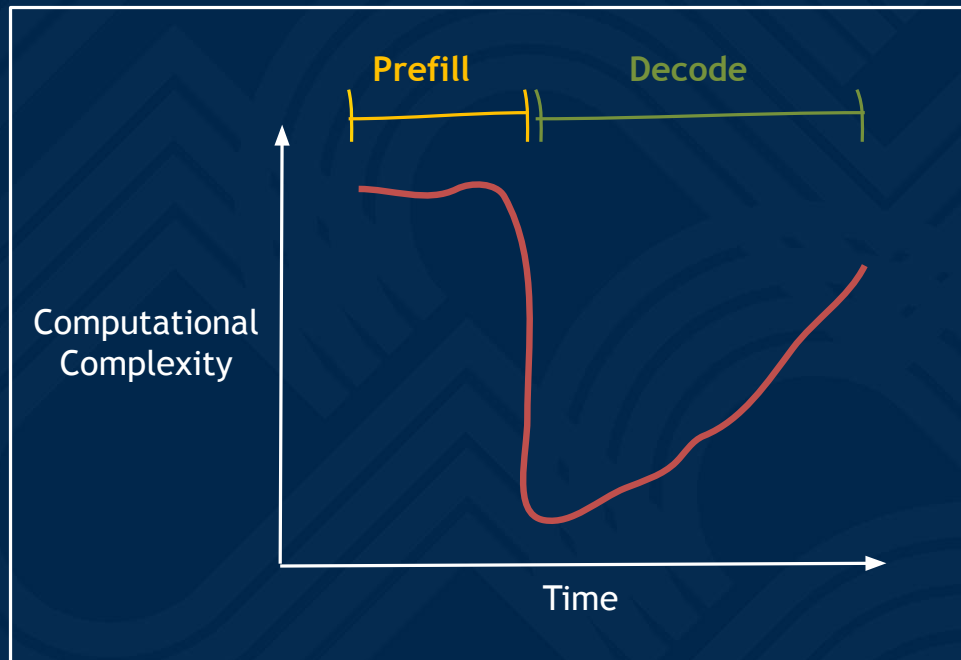
Elastic Instances are the minimum units of execution

2 main types of elastic scaling:

- **Scale down** after prefill
- **Scale up** during decode

1 additional:

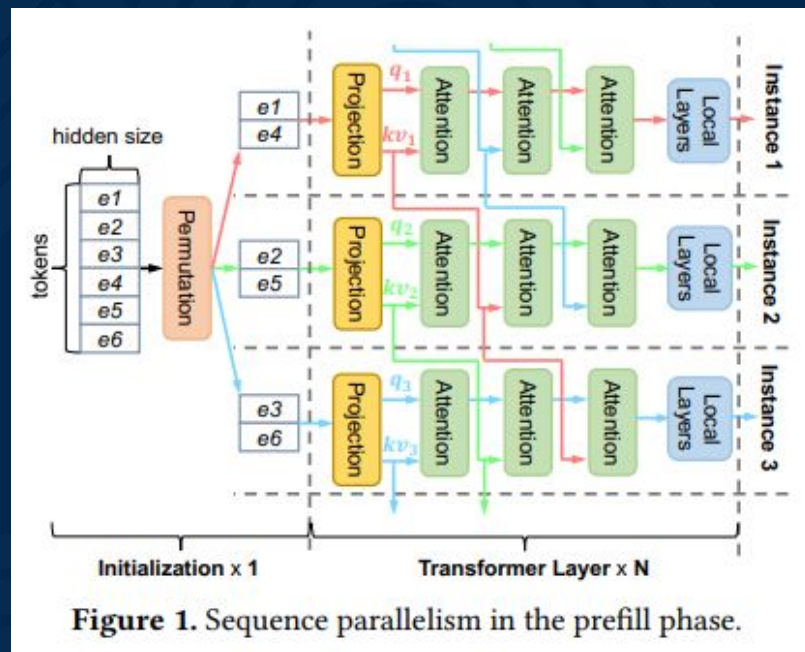
- Optional scale down during decode to save resources



Zero Overhead Elastic Scaling

Proactive Scale-Down

- solves **zero overhead elastic scale-down**
- reuses existing communication mechanisms
 - Sequence Parallelism already sends key-value tensors



Zero Overhead Elastic Scaling

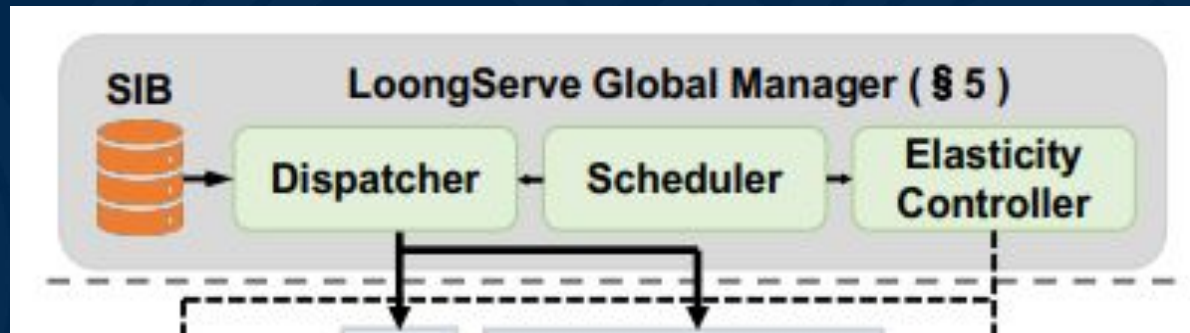
Distributed decoding

- solves zero overhead elastic scale-up
- Single Master Distributed Decoding:
 - Single master leads decode phase of a request
 - Other instances help with computing attention layers and send results to master
 - Other instances don't have to store KV cache so no migration
- Multi-master is an optimization to avoid being resource bound by 1 master

Global Manager

4 components:

- Scaling Information Base (SIB)
- Dispatcher
- Scheduler
- Elasticity Controller



Global Manager - Dispatching

Dispatching Goal: send requests to begin the prefill phase (in FIFO order)

- Determine the number of requests that can be dispatched this iteration

Constraints:

- GPU memory constraint: dispatching should not cause eviction
- GPU compute constraint: maintain tipping point between compute-bound and mem-bound

Global Manager - Elastic Instance Allocation

After determining which requests to dispatch,

Goal: allocate elastic instances to serve those requests

Allocation strategy:

1. Allocate idle instances
2. Allocate instances with smallest KV caches
3. Repeat until Cost > Gain

$$\text{Gain} = \sum_{r \in R_p} \frac{T(R_p, E_p) - T(R_p, E_p \cup e_{min})}{r.\text{input_len}} \quad (3)$$

$$\text{Cost} = \sum_{r \in R_p} \frac{V(e_{min})}{\text{avg_bandwidth} \cdot r.\text{input_len}} \quad (4)$$

$$f[i][k] = \min_{\substack{0 \leq j \leq i, 0 \leq l \leq k, \\ D[j,i] \leq V[l,k]}} (f[j][l] + T(R[j,i], E[l,k])) \quad (5)$$

Global Manager - Batching

Previously, we've determined which requests to dispatch and which elastic instances to use

Batching Goal: assign individual elastic instances to individual requests

Batching Strategy:

1. Group requests by input sequence length
2. Group instances by location & unused KV cache slots
3. Run equation to minimize latency

Global Manager - Elastic Scaling Plan

Scaling Down:

- done proactively, because decode does not benefit from more DoP
- Use minimum # elastic instances that doesn't run out of memory

Scaling Up:

- done when GPU compute / memory becomes insufficient
 - detects this by profiled batch size threshold

When instances receive new scaling plans:

- instances complete their current iteration and reassign in the next

Evaluation - Experimental Setup

Basic Idea: Run an LLM with different serving systems on different datasets

- Model: LWM-1M-Text (~ Llama-2)
- Hardware: 1 server with 8 GPUs
- Datasets:
 - 3 datasets (ShareGPT, L-Eval, LV-Eval)
 - **Mixed dataset** by combining above 3 datasets
- Baseline: Compared against 4 other serving systems
 - vLLM
 - DeepSpeed-MII
 - LightLLM w/ SplitFuse
 - DistServe

Evaluation - Metrics

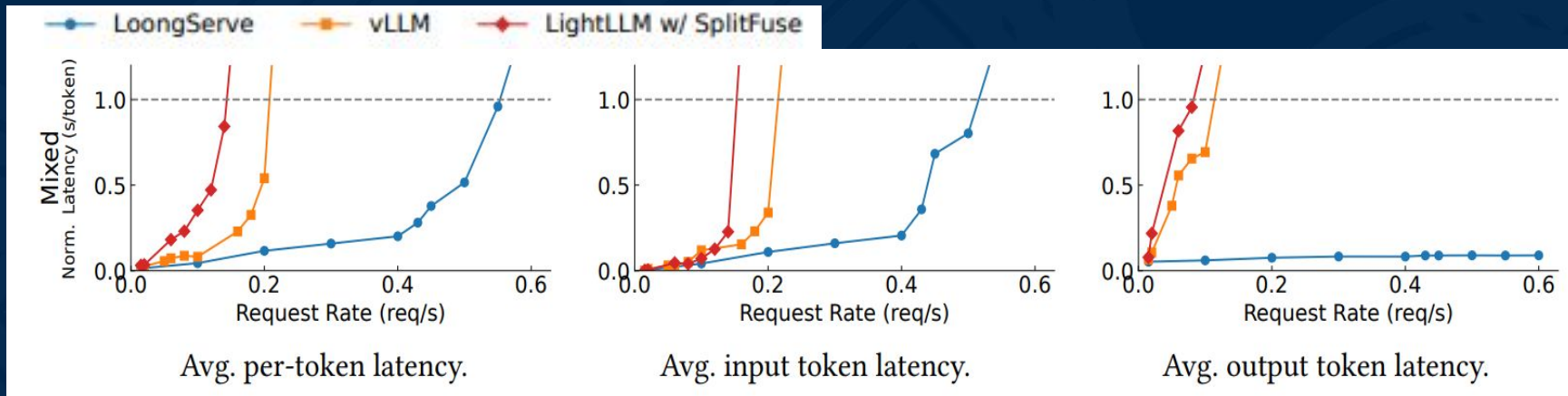
3 key evaluation metrics:

- Per-token latency
 - End-to-end latency / input length
- Input latency
 - Prefill phase time / input length
- Output latency
 - Decoding phase time / input length

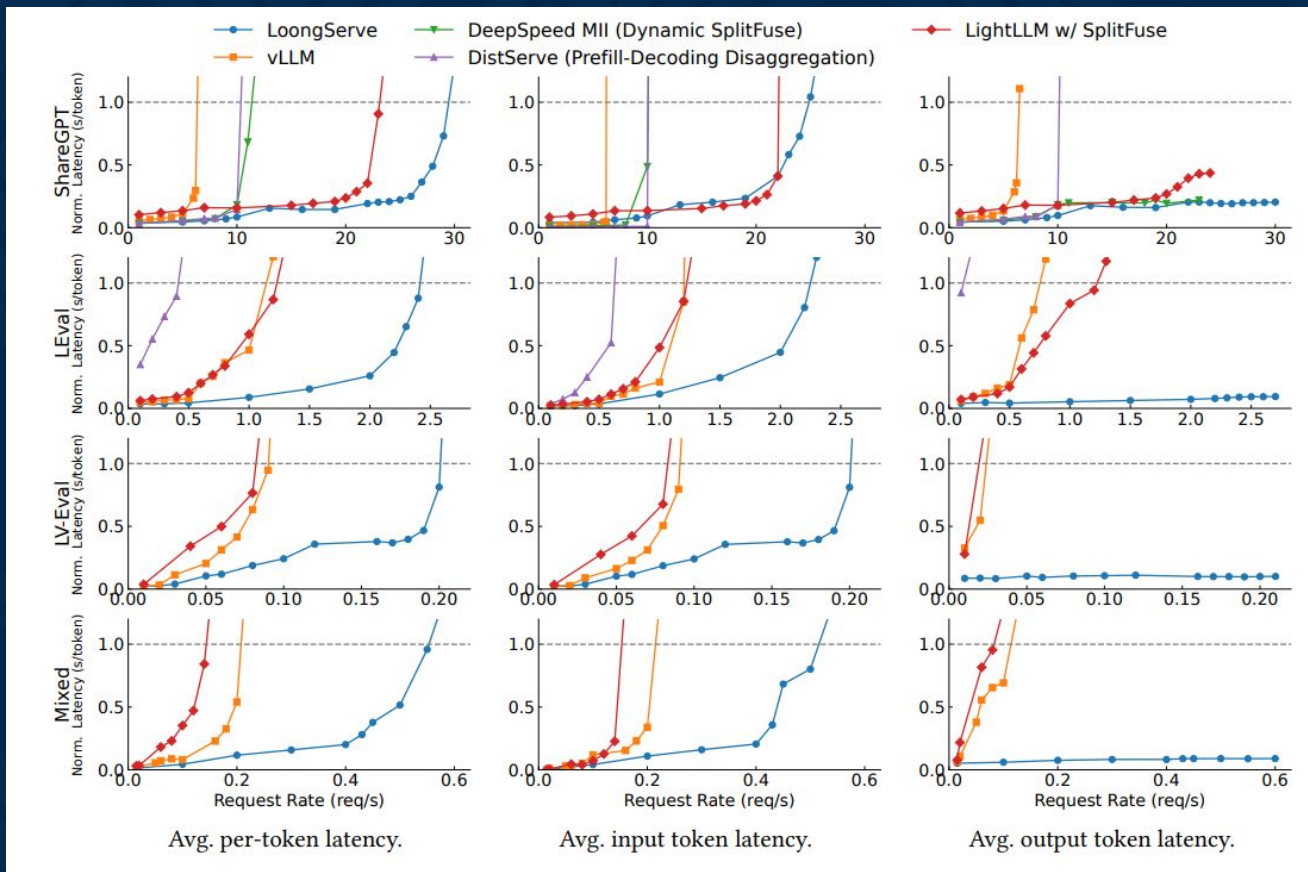
Additionally: P90 Goodput

Evaluation - Mixed Dataset

- Total throughput improves by ~5x (4.64, 3.85, 5.81)



Evaluation - Effect of Increasing Input Length



Evaluation - Additional Metrics

- Multi-node performance: Throughput improves by **~2x**
- P90 Goodput: Improves by **~2x**
- Elastic scale-up:
 - **2.87x** throughput improvement
 - Up to 10% overhead
- Elastic scale-down:
 - **2x** throughput improvement
 - 2% overhead



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

MegaScale-Infer

Serving Mixture of Experts at Scale with
Disaggregated Expert Parallelism

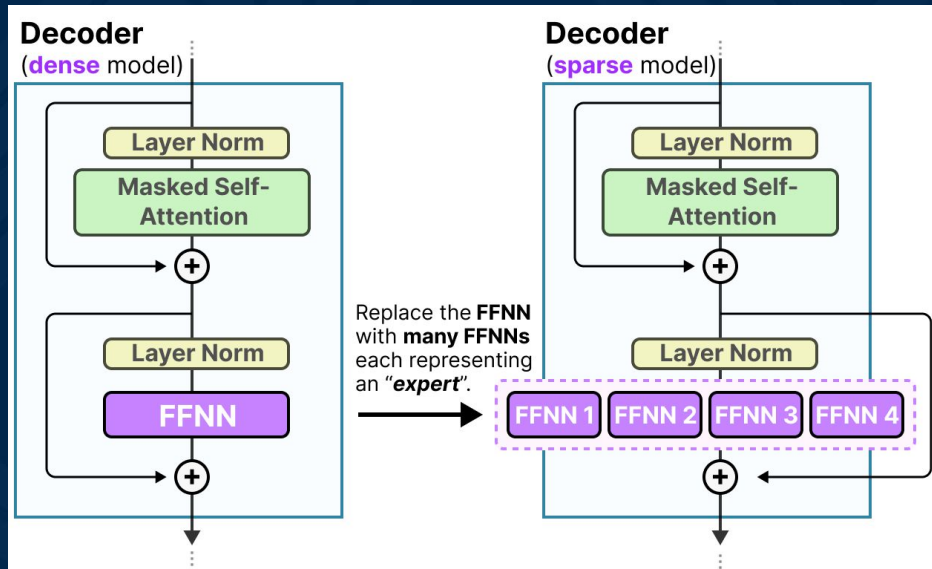
Mixture-of-Experts (MoE)

Mainstream architecture for large-scale LLMs.

Key idea: Make each expert focus on predicting the right answer for a subset of cases

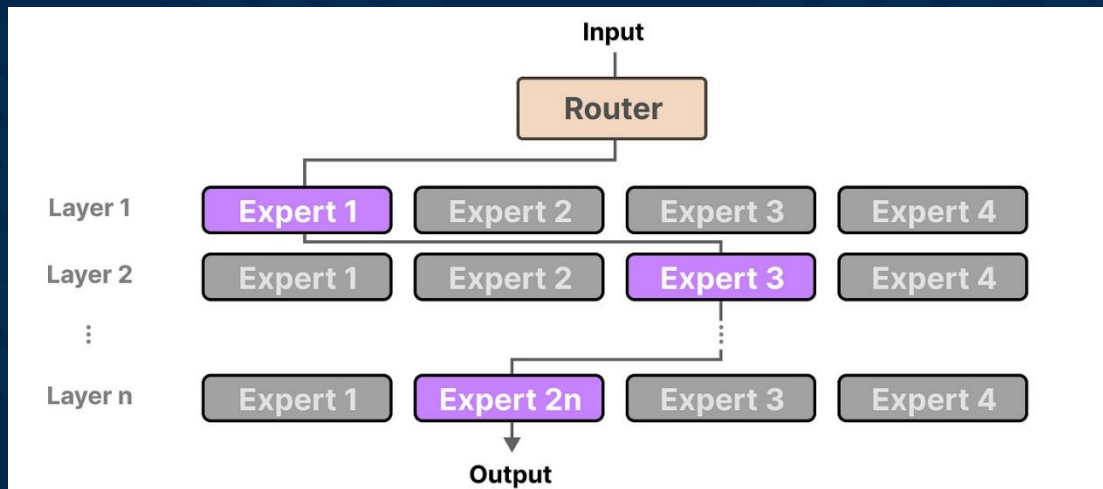
The FFN is replaced by multiple experts.

Routing activates only top-k expert per input token.



Mixture-of-Experts (MoE)

- Final Output: Weighted sum of selected experts' outputs.
- Enables sub linear scaling of FLOPs as model size increases, without compromising quality.



Does lower Computational Complexity mean lower Computational Cost ??

MoE reduces FLOPs — but has communication and utilization bottlenecks

So, the question becomes:

How do we serve xillion-parameter sparse models across thousands of GPUs without overwhelming the network ?

How to Achieve Efficient MoE Inference

Motivation: Prefill and Decode have fundamentally different hardware characteristics.

Prefill Decode Disaggregation: Execute the prefill and decode in different phases on different nodes.

Avoids phase interference and allows us to optimize each phase based on its workload.

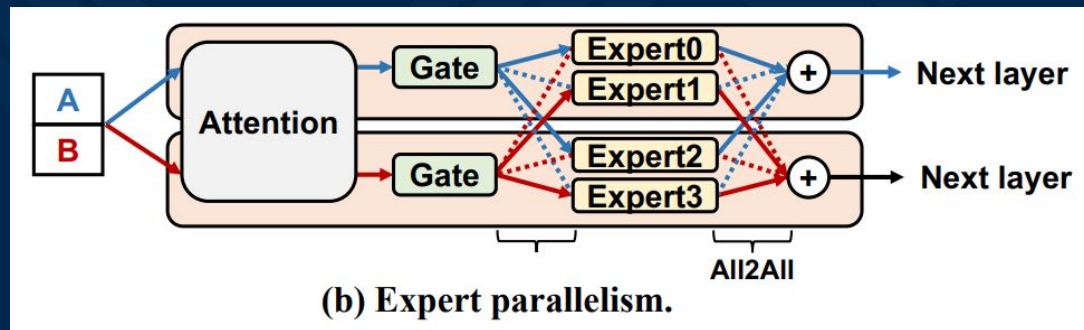
When to use ?

- Long generations
- Imbalanced phases
- Cost optimization
- High decode load
- Batch efficiency

How to Achieve Efficient MoE Inference

Expert Parallelism: Partition the parameters to different nodes at granularity of experts.

Tokens must be sent to experts located on different nodes. Outputs gathered back.



This introduces many to many (M2N) token exchange between attention and expert nodes.

Autoregressive Generation of LLMs & MoE

	Prefill	Decode
Attention	Compute Intensive	Memory Intensive
FFN	Compute Intensive	Memory Intensive to Compute Intensive as batch size increases
Experts	Compute Intensive	Reduced Compute Intensity

MoE Inference: Issues in Decoding Phase

- [Attention] : Memory Intensive nature of Attention cause Low GPU Utilization.
- [FFN]: **Sparsity in MoE** causes small per-expert batch size, hence low compute intensity.

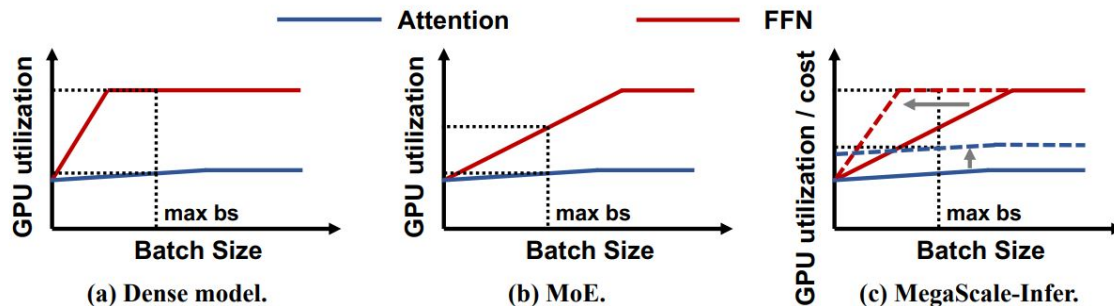
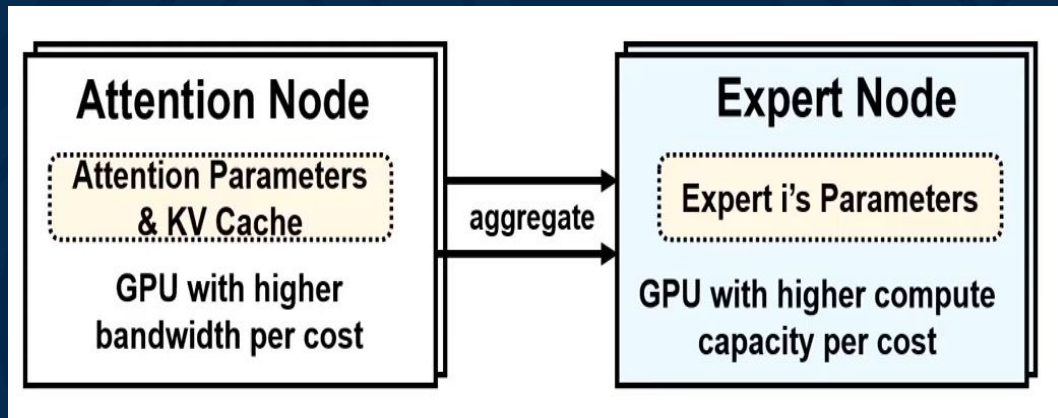


Figure 1 GPU utilization of Attention and FFN vs. batch size in dense model, MoE, and MegaScale-Infer during decoding.

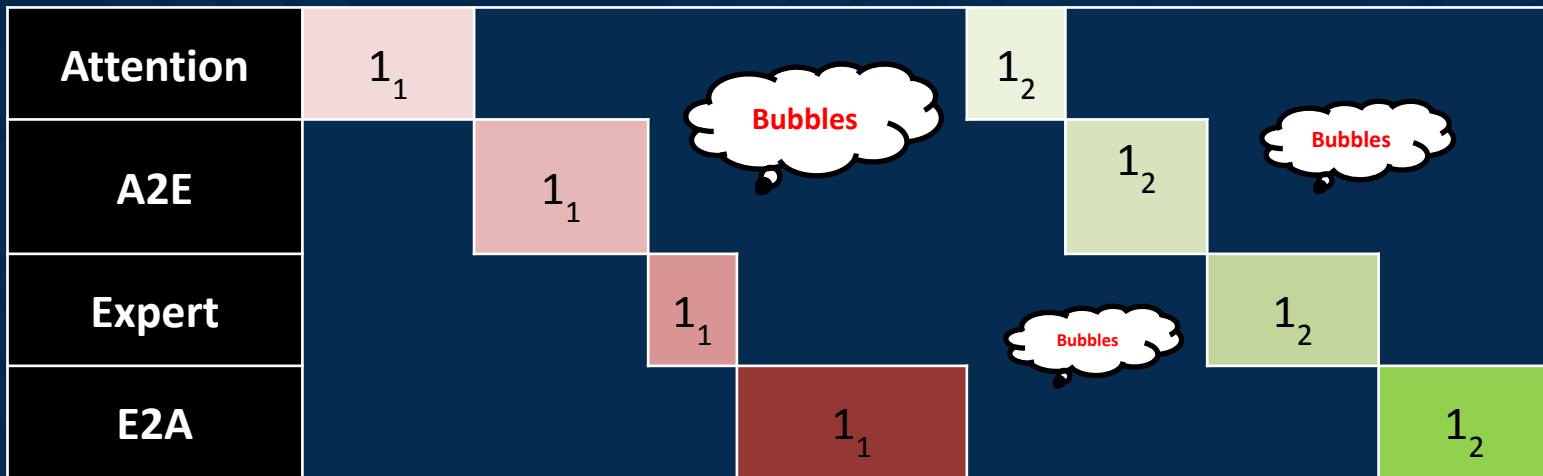
Initial Approach: Attention-FFN Disaggregation

- **Independent Scaling:** Replicate attention modules to increase decoding batch size per expert for FFN modules.
- **Heterogeneous Deployment:** Optimize cost by deploying specific hardware for each model.



Challenge: Dependency leads to Idle Resources

- Due to sequential nature of batch computation, only a portion of resources can be used in parallel.
- Idle time while waiting for outputs to be transmitted over network.



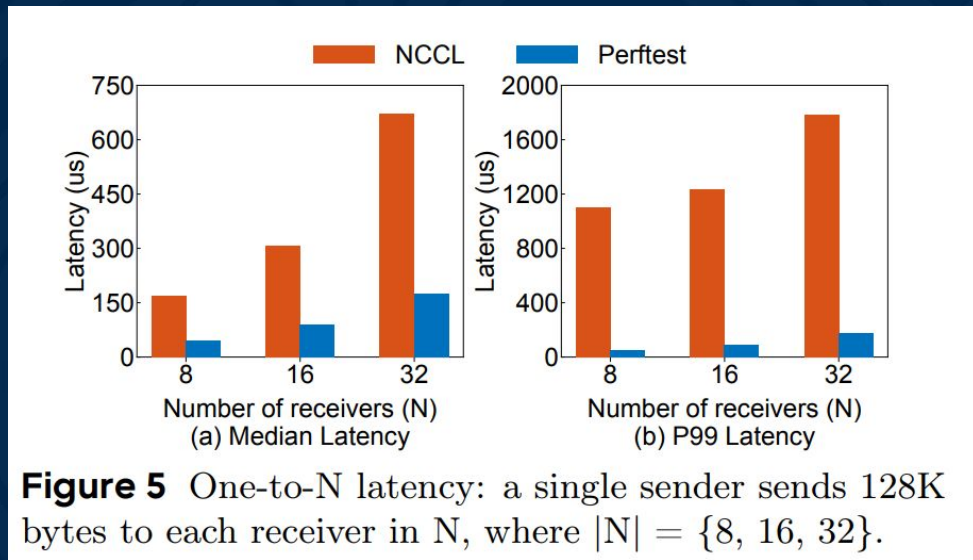
Challenge: High Performance M2N Communication

Independent Scaling requires M2N and N2M communication with M attention GPUs and N expert GPUs.

Existing libraries result in significant performance degradations.

Problems with NCCL:

- Designed for collectives, not sparse M2N
- Data Copy overhead
- Group synchronization overhead.
- Instability at higher percentiles





Mega Scale Infer

Disaggregate
Expert Parallelism

Decouple
attention and
expert modules.
Assign them to
separate GPUs

Ping Pong Pipeline
Parallel

Alternate
partitions to
eliminate
pipeline bubbles

High Performance
M2N Library

Accelerate
many-to many
token exchange
across GPUs

Disaggregate Expert Parallelism

- Architectural Separation:
 - Attention Pool (M nodes) replicate KV cache
 - Expert Pool (N nodes) host distributed experts
- Parallelism Strategy
 - Expert Parallelism across expert GPUs
 - Tensor Parallelism within each pool

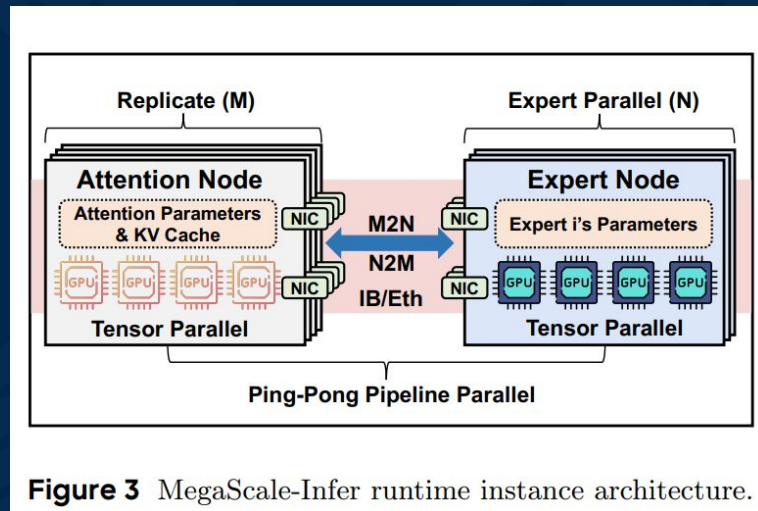


Figure 3 MegaScale-Infer runtime instance architecture.

Ping Pong Parallel Pipeline

Overlap communication with computation through multiple micro batches.



—————▶ Time

Ping Pong Parallel Pipeline

Overlap communication with computation through multiple micro batches.



→ Time

Ping Pong Parallel Pipeline

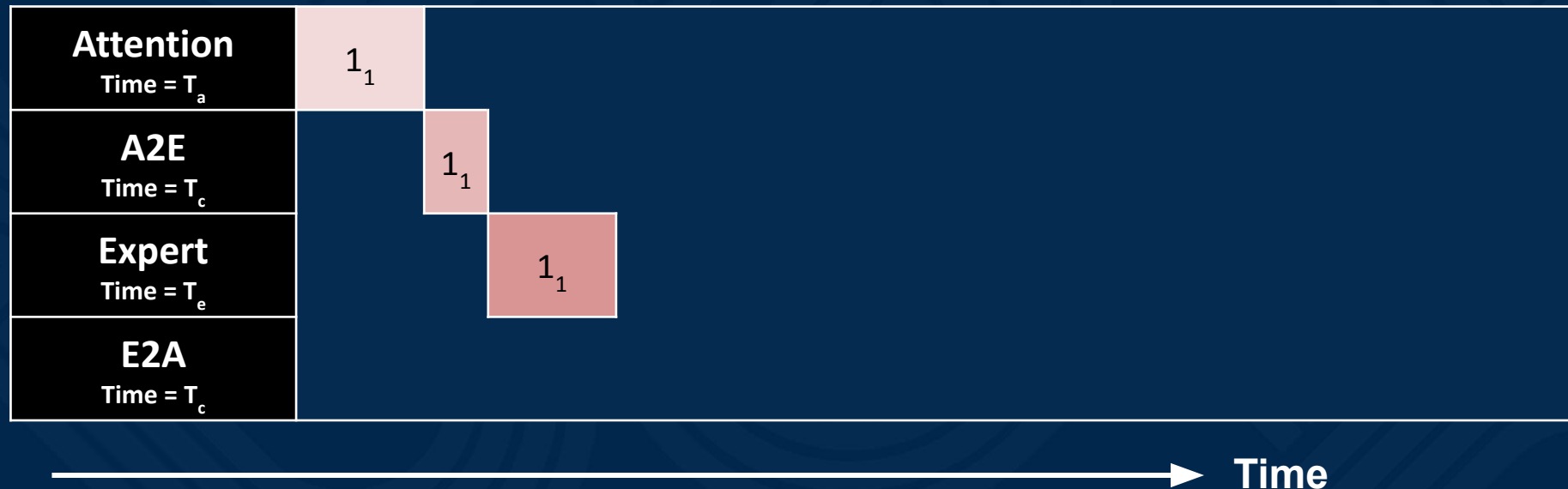
Overlap communication with computation through multiple micro batches.



→ Time

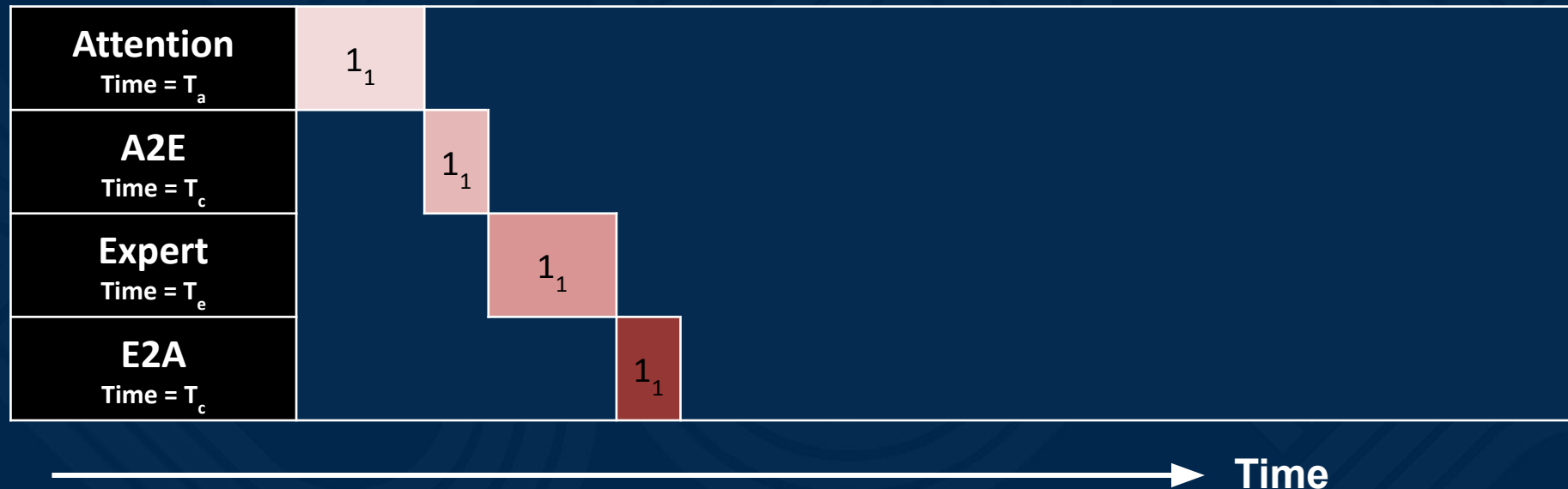
Ping Pong Parallel Pipeline

Overlap communication with computation through multiple micro batches.



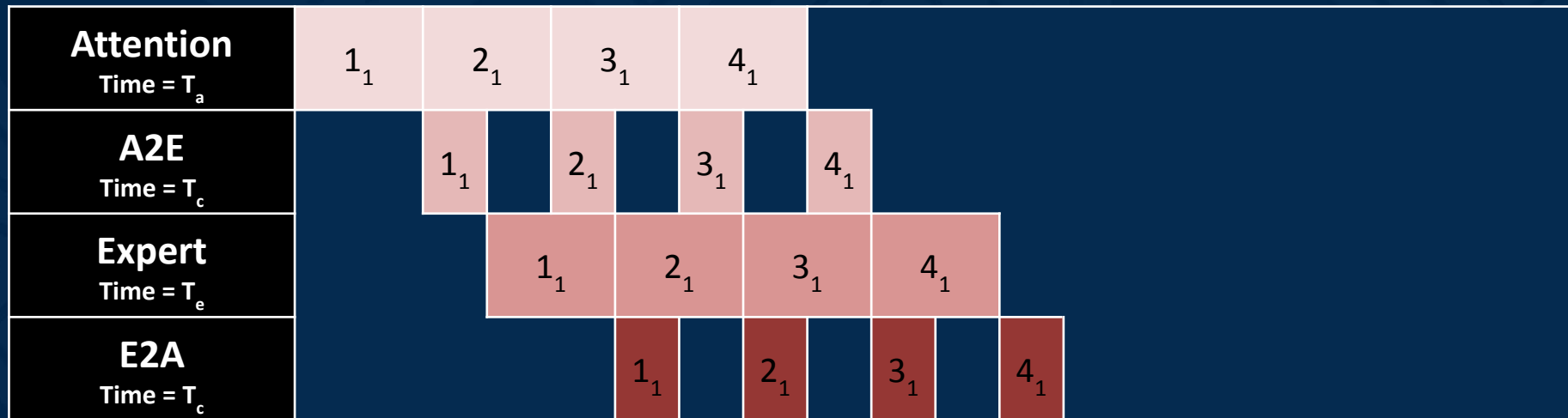
Ping Pong Parallel Pipeline

Overlap communication with computation through multiple micro batches.



Ping Pong Parallel Pipeline

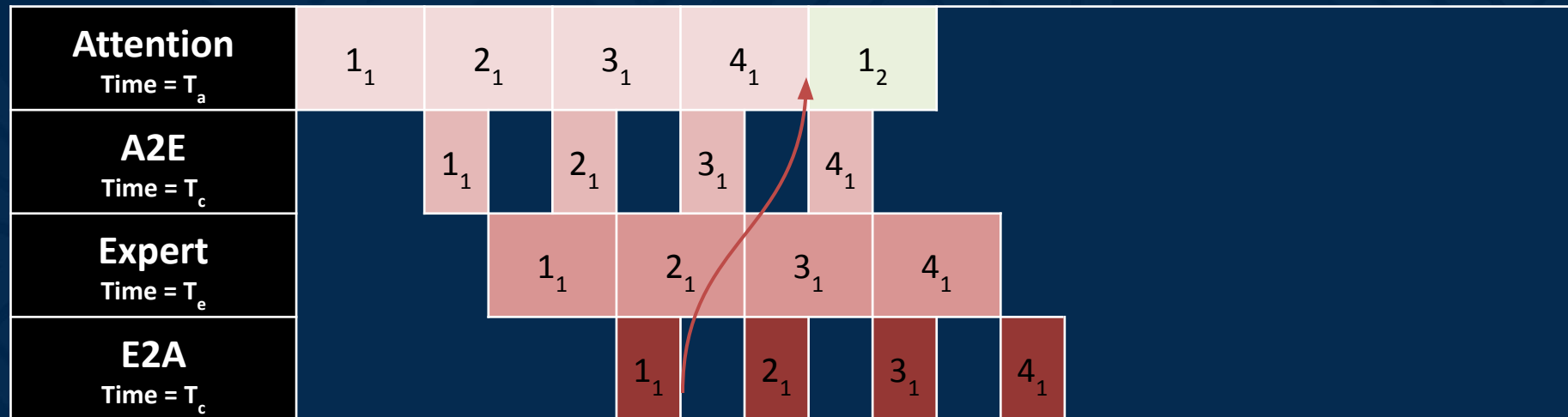
Overlap communication with computation through multiple micro batches.



Time

Ping Pong Parallel Pipeline

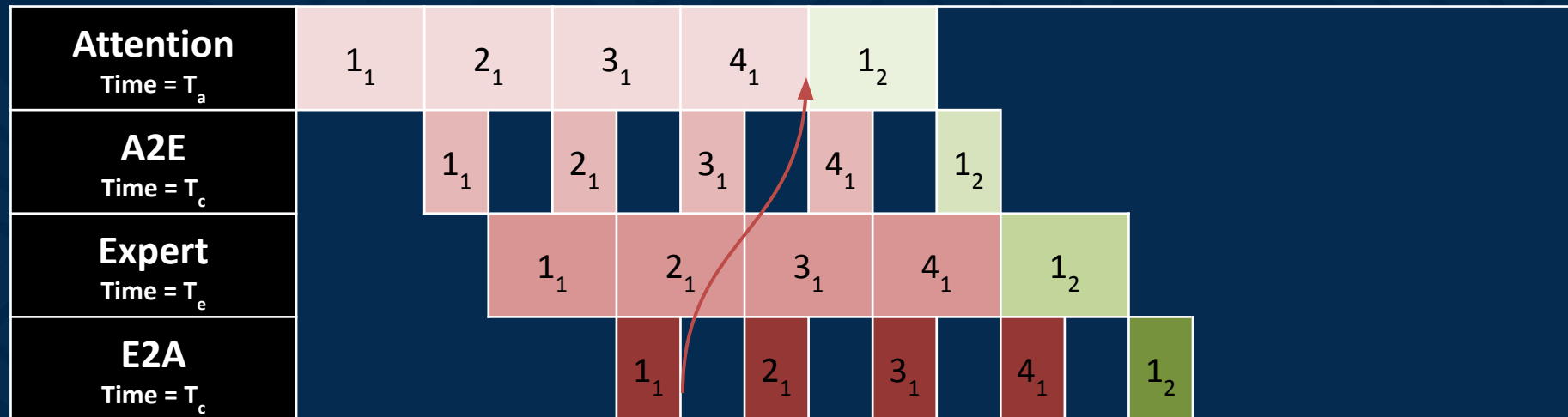
Overlap communication with computation through multiple micro batches.



Time

Ping Pong Parallel Pipeline

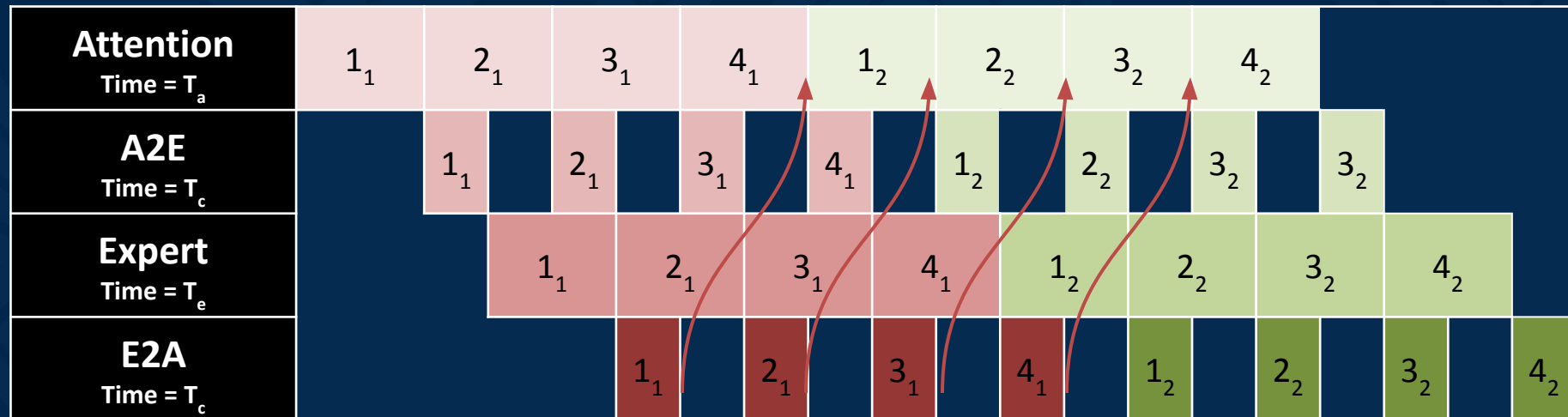
Overlap communication with computation through multiple micro batches.



Time

Ping Pong Parallel Pipeline

Overlap communication with computation through multiple micro batches.



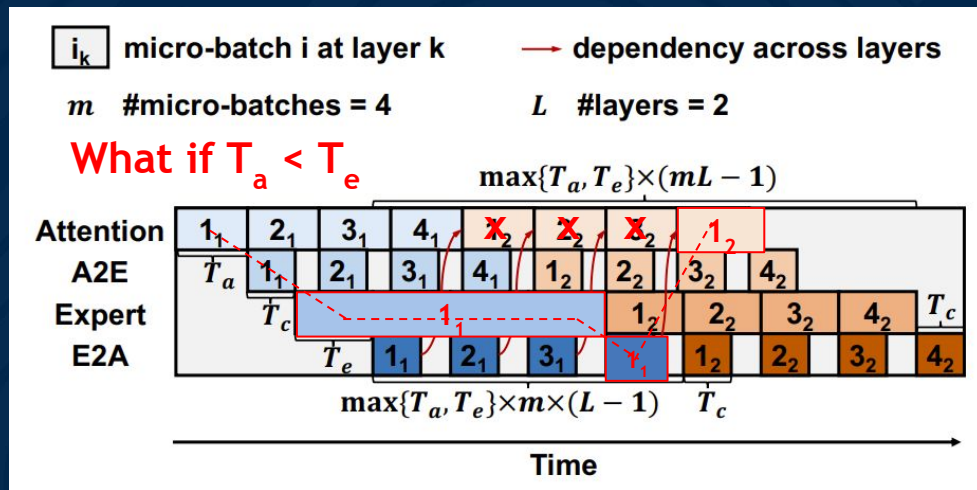
Time

Ping-Pong Pipeline Parallelism

Objective: Reduce GPU idle time

Requirements:

- $T_a \approx T_e$ (Balance attention and expert)



T_a : attention T_c : communication m : # of batch

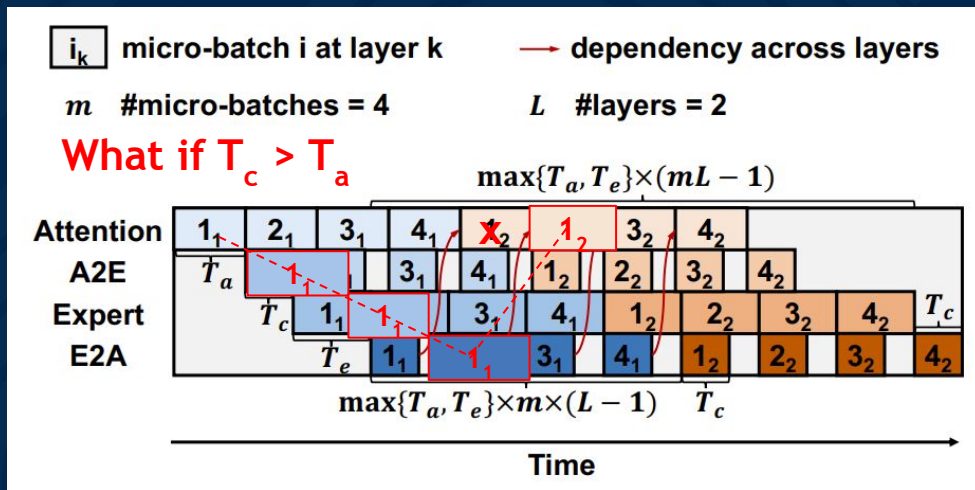
T_e : expert T_f : $\max\{\text{attention}, \text{expert}\}$

Ping-Pong Pipeline Parallelism

Objective: Reduce GPU idle time

Requirements:

- $T_c < T_a$ (Hide communication)



T_a : attention T_c : communication m : # of batch

T_e : expert T_f : $\max\{\text{attention, expert}\}$

Ping-Pong Pipeline Parallelism

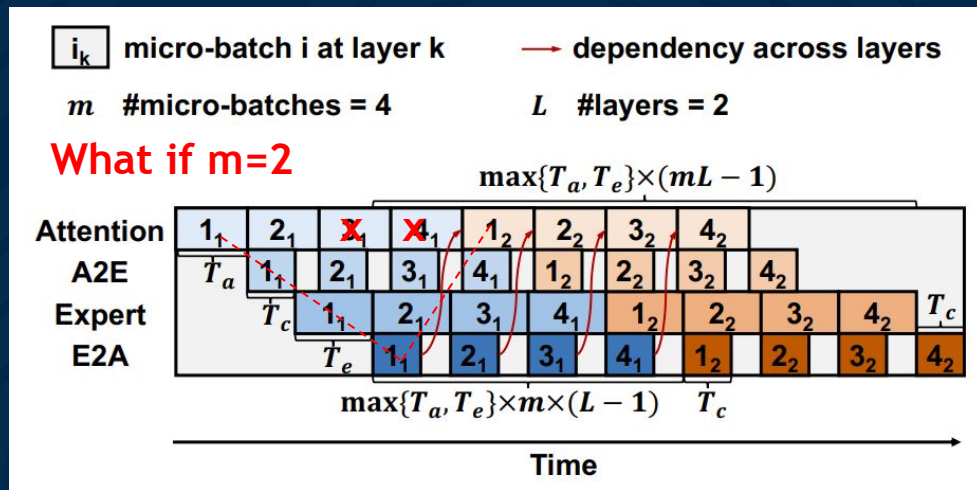
Objective: Reduce GPU idle time

Requirements:

- $m \geq 2 \times (1 + T_c/T_f)$ (Enough batches)

$m \geq 3$ for fast communication $T_c/T_f \leq 1/2$,
pipeline stage latency $\leq 3T_f$

$m \geq 4$ for slow communication $T_c/T_f > 1/2$,
pipeline stage latency $> 3T_f$



T_a : attention T_c : communication m : # of batch

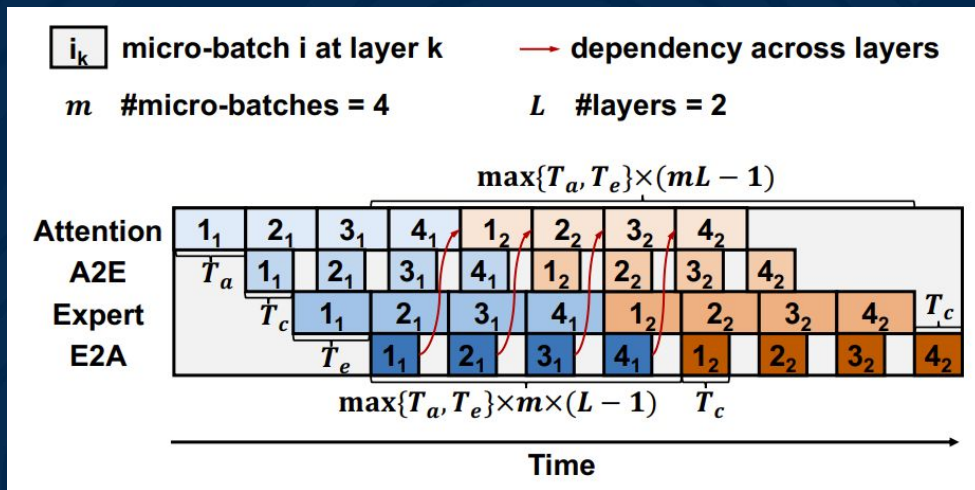
T_e : expert T_f : $\max\{\text{attention, expert}\}$

Ping-Pong Pipeline Parallelism

Objective: Reduce GPU idle time

Takeaways:

- $T_a \approx T_e$ (Balance attention and expert)
- $T_c < T_a$ (Hide communication)
- $m \geq 2 \times (1 + T_c/T_f)$ (Enough batches)



T_a : attention T_c : communication m : # of batch

T_e : expert T_f : $\max\{\text{attention, expert}\}$

Deployment Plan Search

Objective: Maximize throughput per cost while meeting SLOs

1. Iterate over TP size
2. Memory constraints
3. Balance # of Attention Node
→ $T_a \approx T_e$ (Balance attention and expert)
4. Simulate best micro-batch size for SLOs
subject to $m \geq 2 \times (1 + T_c/T_f)$ (Enough batches)

C_a, C_e : memory capacity P_a, P_e : parameter size

N_a, N_e : Max of TP size N_m, N_e : Max of batch size

Algorithm 1 Deployment Plan Search for Decoding Phase

Input: MoE model $G, C_a, C_e, N_m, M_a, M_e$

Output: the optimal deployment plan $plan^*$

```

1:  $plan^* \leftarrow \emptyset$ 
2: for  $tp_e \in \{1, 2, \dots, M_e\}$  do
3:   for  $tp_a \in \{1, 2, \dots, M_a\}$  do
4:     if  $tp_a \times C_a > P_a$  and  $tp_e \times C_e > P_e$  then
5:        $n_a \leftarrow \text{BALANCE}(G, tp_a, tp_e)$ 
6:       for  $m \in \{3, 4, \dots, N_m\}$  do
7:          $plan \leftarrow \{(tp_e, E), (tp_a, n_a), m\}$ 
8:          $B, tpuc \leftarrow$ 
9:            $\text{SIMULATE}(G, plan, SLO)$ 
10:         $plan \leftarrow plan \cup \{B, tpuc\}$ 
11:        if  $plan^*.tpuc < plan.tpuc$  then
12:           $plan^* \leftarrow plan$ 
  
```

Heterogeneous Deployment

- Higher memory bandwidth (H20) for Memory Bound Attentions
- Higher FLOPS (L40S) for Computation Bound FFN Experts

Accelerator	Price	Cap. (GB)	Bw. (GB/s)	Comp. (TFLOPS)	Performance per Cost		
					GB	GB/s	TFLOPS
L20	1.00	48	864	119.5	48	864	119.5
H800	5.28	80	3430.4	989	15.2	649.7	187.3
A800	2.26	80	2039	312	35.4	902.2	138.1
H20	1.85	96	4096	148	51.9	2214.1	80.0
L40S	1.08	48	864	362	44.4	800.0	335.2

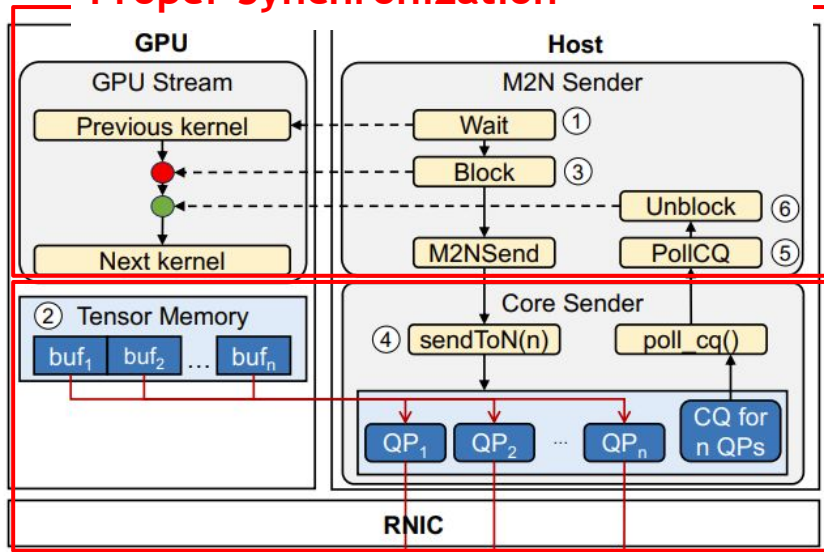
High Performance M2N Communication Library

Recall the challenges of M2N communication: “NCCL is bad”

- Data Copy to CPU Host Creates Overhead
- P2P Group Operation Lacks of Scale
- Additional Group Operation Overhead etc.

High Performance M2N Communication Library

Proper Synchronization



RDMA: GPU-to-NIC copies

Figure 6 M2N Sender components and interactions.

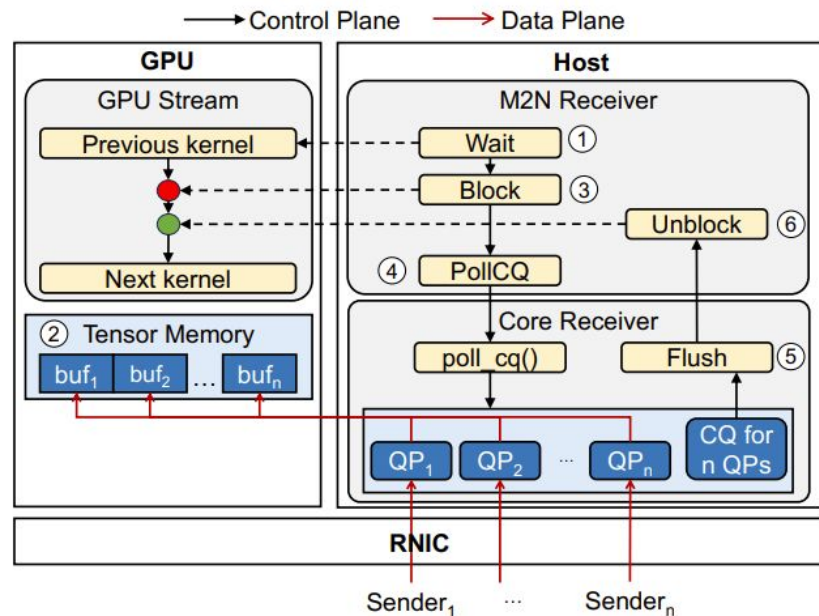


Figure 7 M2N Receiver components and interactions.

Evaluation Setup

- Testbed:
 - Homogeneous Cluster: NVIDIA 80GB Ampere
 - Heterogeneous Cluster: NVIDIA H20 and L40S
- Models
 - Mixtral-8x22B
 - DBRX
 - Scaled-MoE
- Baseline
 - vLLM
 - TensorRT-LLM

Results: Homogeneous Cluster

Improves Decoding Throughput per GPU by up to **7.11x** and **1.90x** compared to baselines.

Reason: Disaggregated Expert Parallelism increases batch size per expert and GPU utilization

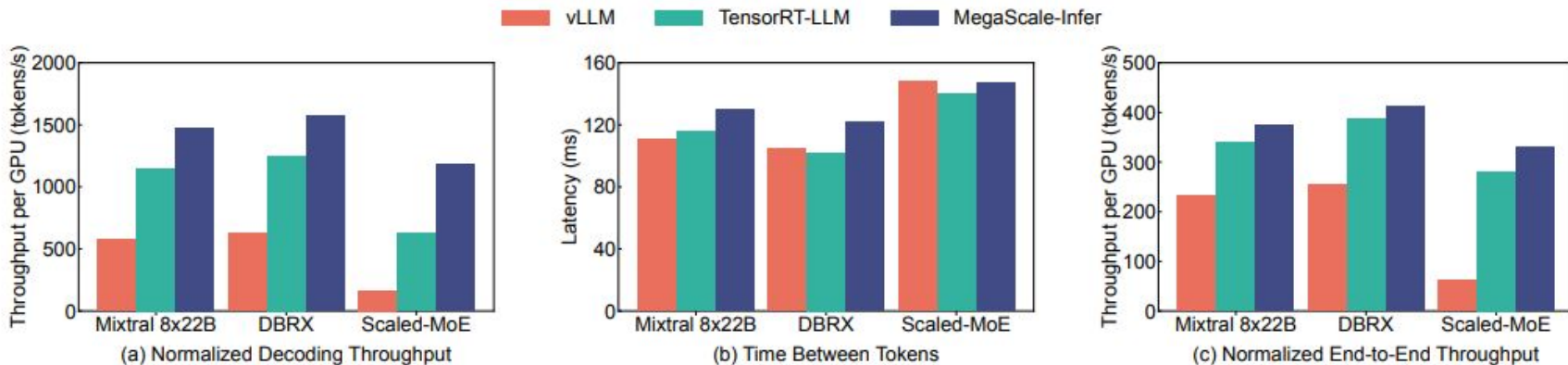


Figure 8 Performance of MegaScale-Infer on NVIDIA Ampere GPUs.

Results: Heterogeneous Cluster

Improves Decoding Throughput per GPU by up to **3.24x** and **1.86x** compared to baselines.

Reason: Cost-effective choice of heterogeneous GPUs

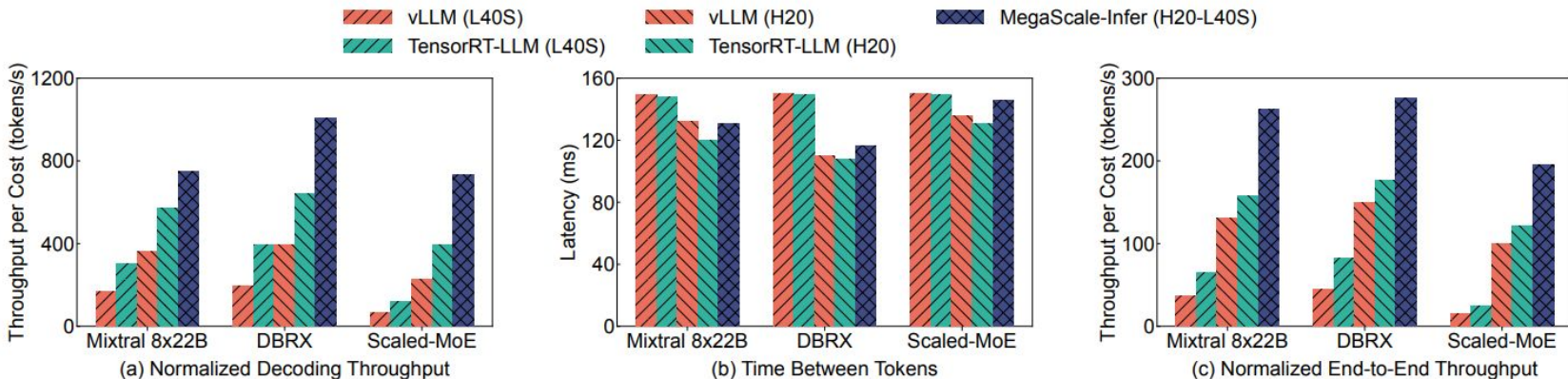


Figure 9 Performance of MegaScale-Infer on NVIDIA H20 and L40S GPUs.

Results: M2N Library Performance

Better Median Latency and Tail Latency, fully utilizing bandwidth

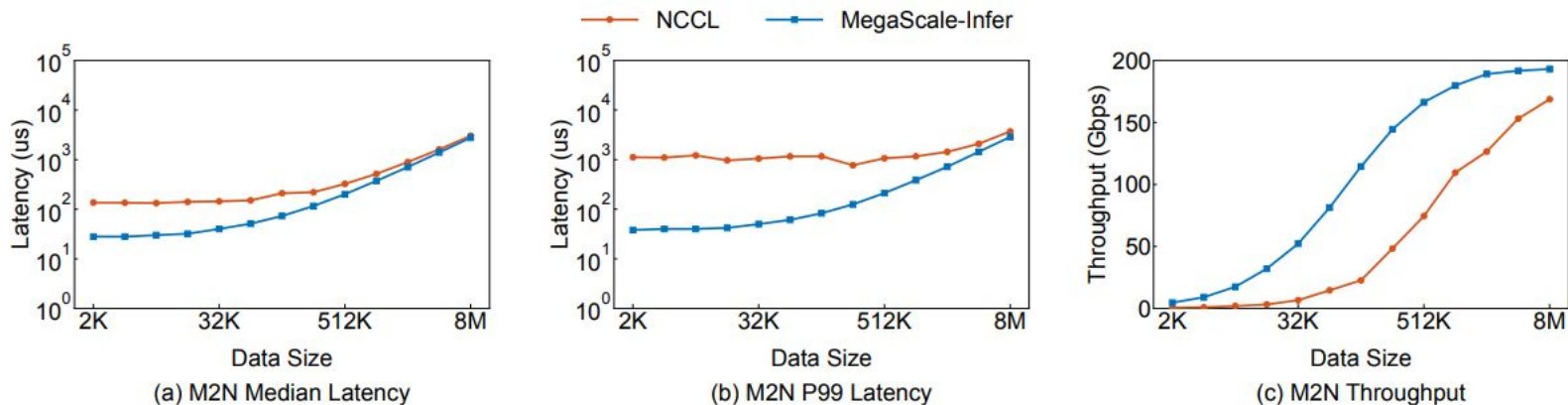


Figure 11 Performance of M2N communication under different data sizes.

Ablation Study

Number of micro-batches

- $m=3$ is enough for most models

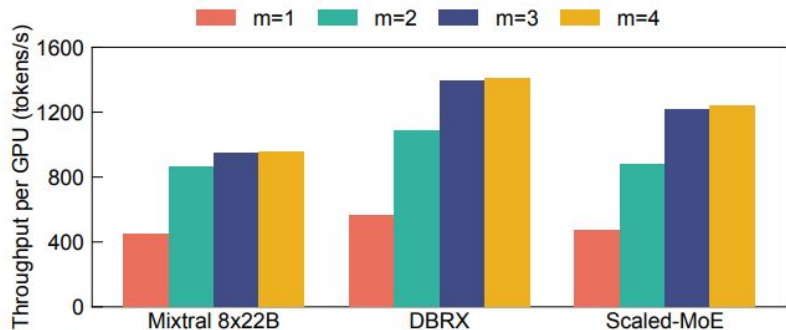


Figure 14 Normalized decoding throughput under different numbers of micro-batch.

Number of attention nodes

- 8 attention nodes is the balance

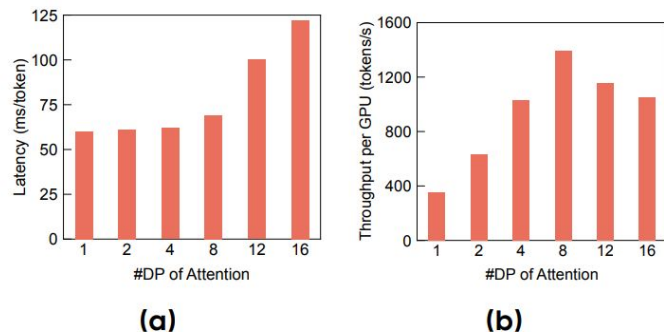


Figure 15 Performance of DBRX under different DP degree of attention. (a) Per output token latency. (b) Decoding throughput normalized by number of GPUs.

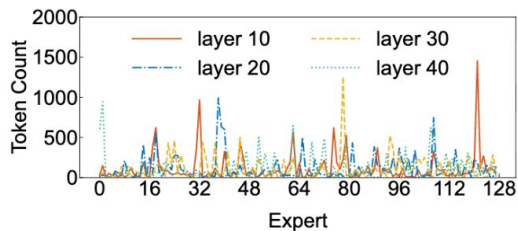
Deployment Experience

Expert Balance

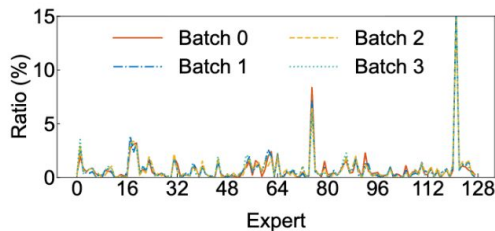
- Load imbalance among experts
- Decode loads are stable among experts
- Prefill loads are various among experts

Attention Balance

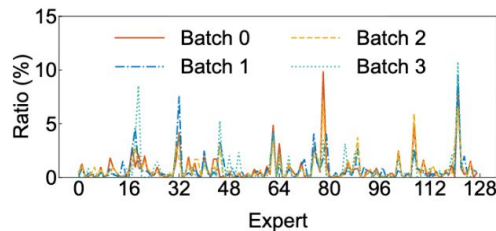
- Difference sequence lengths
- Batch composition to balance latency



(a)



(b)



(c)

Figure 16 Expert load distribution in real traffic. (a) Received token count of each expert in a batch. (b) Received token ratio of each expert during decoding. (c) Received token ratio of each expert during prefill.

Conclusion

Distributed LLM serving requires *dynamic resource allocation* for *varying loads*

LoongServe (Elastic Sequence Parallelism)

- Sequence Parallelism
- Temporal elasticity between prefill, decode, and longer decode phases
- Change DoP in different phases and Optimize KV-cache locality between phases

MegaScale-Infer (Disaggregated Expert Parallelism)

- Mixture of Experts (MoE)
- Spatial disaggregation of attention and expert for independent scaling
- Balance attention and expert nodes and Optimize M2N communication

Thank You!